

House Price Predictor

A Professional Machine Learning Solution for Real Estate Valuation

This repository contains a comprehensive House Price Prediction system, utilizing advanced regression techniques and a streamlined user interface to provide accurate property valuations.



Streamlit Web User Interface

The application features a clean, responsive web interface built using Streamlit. It organizes inputs into logical sections—General Information, Room & Floor Sizes, Rooms/Bathrooms/Garage, and Additional Features—with helpful tooltips and real-time metric updates.



House Price Prediction App

Please provide the details of the property below, and the AI will estimate its market value.

1. General & Building Information

Building Style / Type 	Material & Finish Quality 	Year Built 
20 ▼	5	2000 – +
Total Land Area (sq ft) 	Overall House Condition 	Year Remodeled 
10000 – +	5	2000 – +

2. Room & Floor Sizes

Finished Basement Area (sq ft)	1st Floor Area (sq ft)	Wooden Deck Area (sq ft)
500.00 – +	850.00 – +	0.00 – +



Dataset Information

- **Source Dataset:-**
https://www.kaggle.com/datasets/jenilhareshbhaighori/house-price-prediction/data?select=house_cleaned.csv
- **Target Variable (y):** **SalePrice** (Final selling price of the house).
- **Feature Engineering & Preprocessing:**
 - **Identifier Removal:** Dropped the unique identifier column (**Id**) as it adds no predictive value.
 - **Data Integrity:** Checked for missing values, handled data types, and split the dataset using an 80-20 train-test split (**random_state=42**).
 - **Normalization:** Utilized **StandardScaler** where necessary to normalize feature scales (mean = 0, standard deviation = 1) for distance-sensitive algorithms.



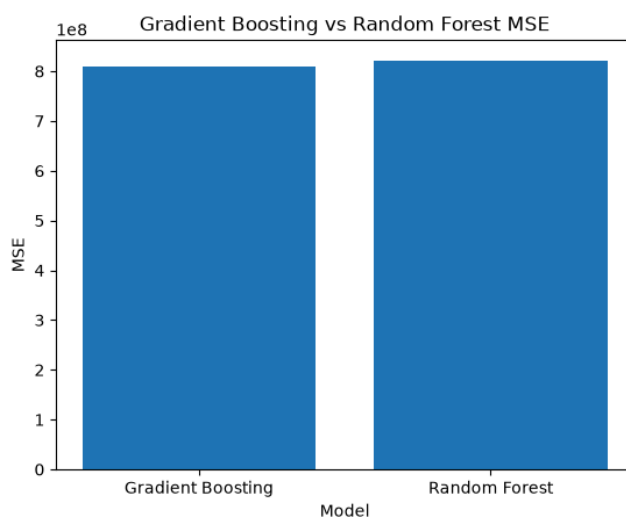
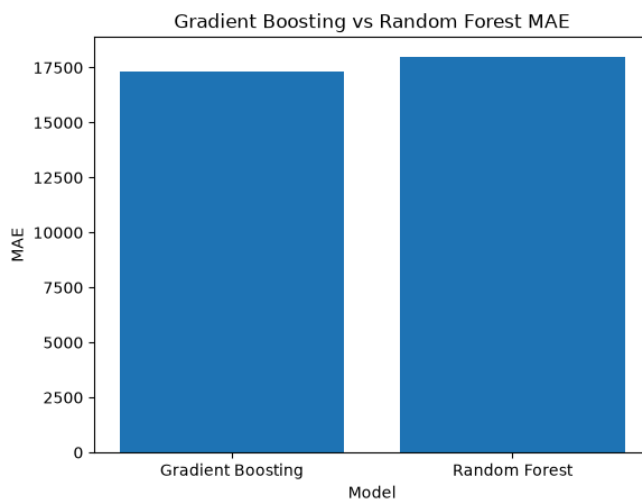
Machine Learning Model & Algorithm Selection

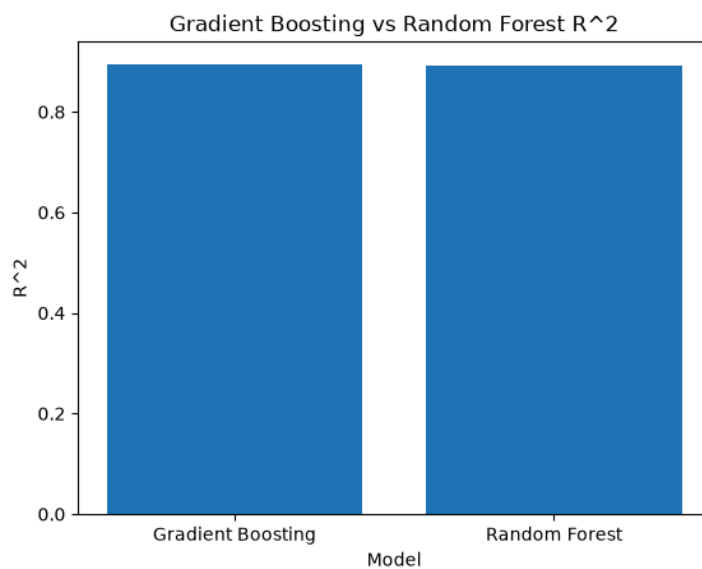
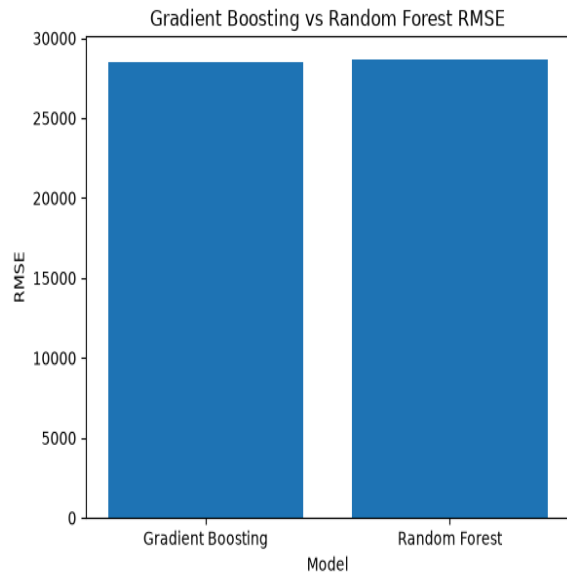
To achieve optimal predictive accuracy, three distinct machine learning regression models were trained, evaluated, and compared:

Model	Technique	Key Characteristics
Linear Regression	Baseline	Understands fundamental linear relationships between features.
Random Forest	Bagging Ensemble	Uses multiple decision trees (n_estimators=300) to capture non-linear patterns.
Gradient Boosting	Boosting Ensemble	Selected Production Model. Sequentially corrects errors from previous trees.

Why Gradient Boosting?

- **Sequential Error Correction:** It builds trees sequentially, where each new tree specifically targets and minimizes the residual errors of the previous trees.
- **Superior Performance on Tabular Data:** Real estate pricing involves complex, non-linear interactions (e.g., how age, square footage, and overall quality compound together). Gradient Boosting models excel at capturing these nuances without requiring heavy manual feature scaling.
- **Hyperparameters:** `n_estimators=300`, `learning_rate=0.05`, `max_depth=3`, `random_state=42`.





Evaluation Metrics & Model Comparison

Models were evaluated using standard regression metrics to ensure reliability:

- **MAE (Mean Absolute Error):** Measures the average magnitude of absolute prediction errors.
- **MSE (Mean Squared Error):** Squares errors, heavily penalizing larger outlier mistakes.
- **RMSE (Root Mean Squared Error):** Interpretable error metric in the exact same units as SalePrice (\$).

- **R² Score (Coefficient of Determination):** Represents the proportion of variance in the target variable explained by the features (closer to 1.0 is better).

All nurmiacal data:-

- R2: 0.894401570332499
- MAE : 17295.32
- MSE : 809974402.52
- RMSE : 28460.05
- R2 : 0.89

Gradient Boosting consistently yielded the highest R² score and lowest error margins compared to baseline and standard random forest configurations.



Tech Stack & Libraries

- **Python:** Core Programming Language
- **Pandas & NumPy:** Data Manipulation & Numerical Operations
- **Scikit-Learn:** Model Training, Metrics, and Scalers
- **Matplotlib:** Data Visualization & Model Comparison Charts
- **Joblib:** Model Serialization
- **Streamlit:** Interactive Web UI Framework



Installation & Local Execution

Follow these steps to run the application on your local machine:

1. **Clone the repository:**

```
git clone <span type="placeholder"
placeholder-type="person"></span>/house-price-predictor
```

2. **Install required dependencies:**

```
pip install -r requirements.txt
```

3. **Run the Streamlit application:**

```
streamlit run CODE/house_price_predictor_streamlit_ui.py
```

Deployment

This project is deployment-ready for **Streamlit Community Cloud**:

1. Push your repository files to GitHub.
 2. Link your repository on the Streamlit Cloud dashboard.
 3. Set the main file path to
`CODE/house_price_predictor_streamlit_ui.py` and deploy!
-

Project Lead: surya prakash Nayak

Contact Information

- **Email:** [surya prakash Nayak](mailto:surya.prakash.Nayak)
- **LinkedIn:-** www.linkedin.com/in/surya-prakash-nayak-9274513a0
- **GitHub:** <https://github.com/suryaprakashn970-prog>